Artificial Intelligence

CSE471s, Spring 2026

Quoridor Game

PREPARED BY:

Name	ID
Omar Samir Salem Mohamed	2300720
Yousef Osama Elsayed Mohamed	2300647
Kevin Hany Beshara	2300381
Mariam Maged Mohammad	2300670
Mohamed Hisham Saad Elsayed	2301257

PREPARED FOR:

- Dr. Manal Morad
- Eng. Robeir Samir

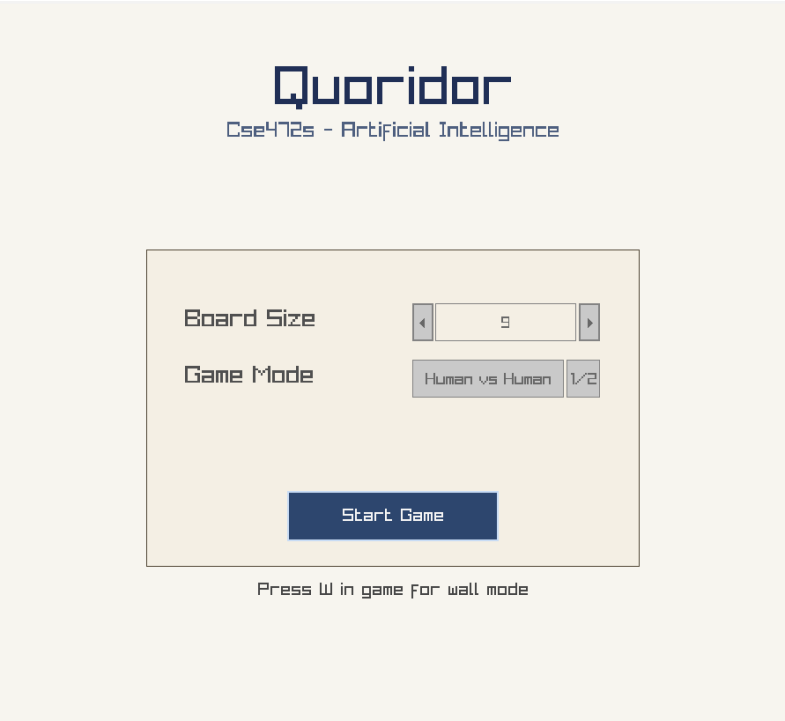
SUBMISSION DATE: 28/5/2026

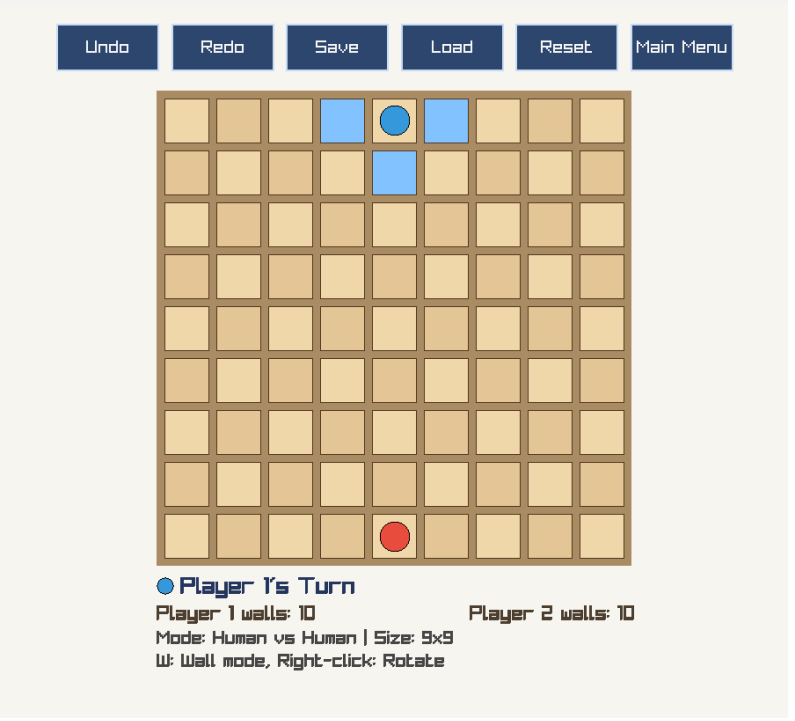
TABLE OF CONTENTS

1. GAME OVERVIEW

Quoridor is an abstract strategy board game invented by Mirko Marchesi and first published in 1997 by Gigamic. It has won several awards including the prestigious Mensa Mind Game award, and is widely studied in artificial intelligence research because of its high branching factor and the strategic tension between advancing your own pawn and blocking the opponent with walls.

In this project, we implemented a full 2-player version of Quoridor in C++ using the Raylib graphics library. The game supports Human vs. Human and Human vs. AI modes, three AI difficulty levels, and several bonus features including game saving and loading, undo and redo, and a configurable board size.





2. DESIGN DECISIONS AND ARCHITECTURE

2.1 HIGH-LEVEL ARCHITECTURE

The project follows a strict Model-View-Controller (MVC) architecture divided into four source directories. This separation ensures that the game rules have no knowledge of Raylib, the renderer has no knowledge of AI algorithms, and the controller acts as a neutral bridge between all layers.

Layer	Directory	Responsibility
Model	src/model/	Pure game data and rules — board, cells, walls, players, moves, and all state transitions. Zero dependencies on Raylib.
Controller	src/controller/	Orchestrates game flow: processes moves, maintains undo/redo stacks, delegates save/load to SaveManager.
AI	src/ai/	Swappable AI strategies (Random, Greedy, Minimax) and the BFS pathfinding utilities they depend on.
UI	src/ui/	All Raylib code: board rendering, HUD, input handling, menu and game screens, and the main application loop.

The key design decision was to keep the model completely free of any rendering or AI code. This means the same GameState object is used by both the renderer and all AI strategies without any modification, and it can be unit-tested without a graphics context.

2.2 MODEL LAYER

The model is built from small, focused classes that each own one piece of state:

GameState — the single source of truth. Holds the board, both players, the current turn, game status, and the full move history. All AI strategies and the renderer read from this object.

GameBoard — a 2D grid of Cell objects. Manages wall placement and validates pawn movement using the wall flags stored on each cell.

Cell — stores four boolean wall flags (North, East, South, West). When a wall is placed, the two cells on each side of the gap have their flags set, so checking if a move is blocked is a simple O(1) lookup.

Wall — a placement record (row, col, orientation). Its conflicts() method covers all four overlap cases: exact duplicate, perpendicular crossing, and same-direction adjacency.

Move — a discriminated union of either a pawn move (target Position) or a wall placement (Wall). The controller and all AI strategies handle both move types through this single type.

2.3 STRATEGY PATTERN FOR AI

AI difficulty uses the Strategy design pattern. IAIStrategy defines one method, selectMove(state, playerId), and three classes implement it. GameApp selects the correct strategy at game-start based on the difficulty setting and injects it into AIPlayer. Changing difficulty at runtime would require only swapping the pointer — no conditional logic is scattered through the game loop.

1.4 UNDO/REDO DESIGN

GameController maintains two move stacks: undoStack and redoStack. Every successful move is pushed to undoStack and clears redoStack. Undo pops a move and calls GameState::undoMove(), which replays the full move history from scratch to reconstruct the previous state. Redo re-applies the popped move.

In Human vs. AI mode, undo pops two moves at once — the human's last move and the AI's reply — so the human always resumes on their own turn. Redo in this mode also clears the redo stack after a single re-application, preventing stale AI replies from being re-inserted.

1.5 SAVE/LOAD

SaveManager serialises the complete GameState (board size, wall placements, player positions and wall counts, current turn, game status) along with both the undo and redo stacks into a binary file with the .qdr extension. On load it deserialises directly back into a GameState object and restores the stacks, so the player can still undo moves made before the save point.

1.6 UI LAYER

The UI is split into five classes. GameApp is the top-level loop that owns all subsystems and switches between the menu and the game screen. MenuScreen uses raygui widgets to collect the board size, game mode, and difficulty. BoardRenderer computes the pixel layout from screen dimensions every frame so the board scales correctly when the window is resized. HudRenderer draws the turn indicator, wall counts, mode/difficulty info, and timed status

messages. InputHandler maps mouse clicks to board positions, manages wall preview orientation, and surfaces one pending Move per frame to the controller.

2. IMPLEMENTATION CHALLENGES AND SOLUTIONS

This section describes the main technical problems we encountered during development and how we solved each one.

2.1 WALL CONFLICT AND PATH-BLOCKING DETECTION

Challenge 1: Walls placed on top of each other

Early on, two walls could be placed with the same anchor point and different orientations, which caused them to visually cross into a plus-sign shape that does not exist in the real game. We initially only checked whether the anchor points were identical, but we missed the case of a perpendicular crossing.

Solution: We added four conflict cases inside Wall::conflicts(): an exact duplicate (same row, col, orientation), a perpendicular cross (same row and col but different orientation), a horizontal overlap (two horizontal walls on the same row one column apart), and a vertical overlap (two vertical walls on the same column one row apart). Once all four cases were covered, illegal placements were fully rejected.

Challenge 2: Off-by-one error in the wall grid

The wall placement grid has (boardSize − 1) × (boardSize − 1) positions, not boardSize × boardSize. A wall at anchor (row, col) spans cells (row, col) and (row, col+1) for a horizontal wall, so the last valid anchor column is boardSize−2. We were initially checking row < boardSize and col < boardSize, which allowed walls to be anchored in the last row or column where they would extend off the board and corrupt cell data.

Solution: We changed the bounds check to row < boardSize−1 and col < boardSize−1 inside GameBoard::placeWall(). We also added the same check to InputHandler::wallFromMouse() so the preview would disappear when the cursor moved into the out-of-bounds groove region.

2.2 PAWN JUMP AND DIAGONAL-JUMP LOGIC

Challenge 1: Diagonal jump directions were wrong

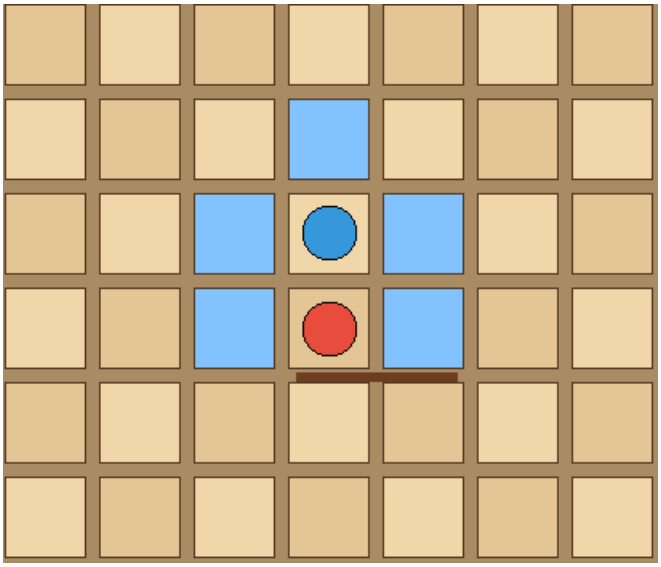
The rule says: if you are approaching the opponent vertically (from above or below), the two diagonal alternatives go left and right. If you are approaching horizontally (from left or right), they go up and down. We initially mixed this up and allowed the wrong diagonals, which let pawns teleport to non-adjacent cells in certain positions.

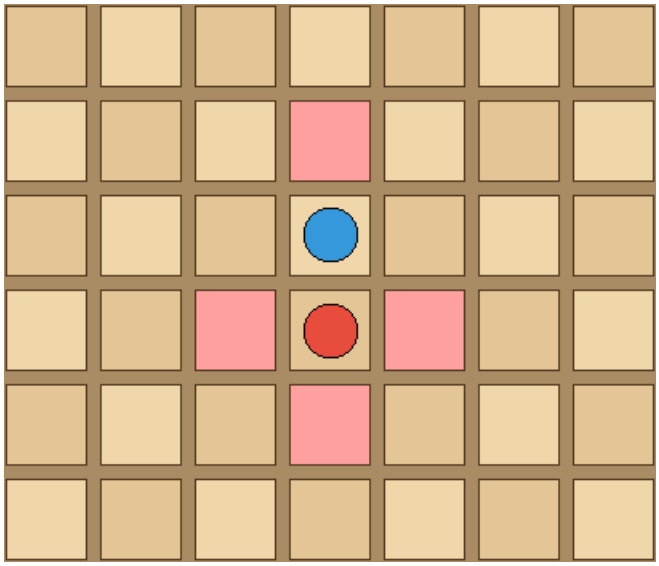
Solution: We added an explicit branch inside getValidPawnMovesByIndex(): if the approach direction is ‘U’ or ‘D’, check lateral moves (‘L’ and ‘R’) from the opponent’s position; if the approach is ‘L’ or ‘R’, check vertical moves (‘U’ and ‘D’). We also verified each lateral/vertical move with board.isValidMove() before adding the diagonal, so a wall blocking the side escape is correctly respected.

Challenge 2: Diagonal jump through a wall

Even after fixing the directions, diagonal moves were sometimes going through walls. For example, a player approaching from above could jump diagonally left even if there was a wall blocking the left side of the opponent’s cell. The rule is that the diagonal is only available if the path to that side of the opponent’s cell is clear.

Solution: Before adding a diagonal destination to the move list, we call board.isValidMove(opponentPosition, lateralDirection). This checks whether the opponent’s cell has a wall on its left or right face (or up/down face for horizontal approach). Only if that check passes do we push the diagonal square as a valid move.





2.3 MINIMAX PERFORMANCE

Challenge 1: Depth 3 made the game unresponsive

When we first tested Minimax at depth 3, the AI took between five and fifteen seconds to respond on a standard 9×9 board. Quoridor has a very high branching factor because each turn can produce up to roughly 130 valid moves (4 pawn moves plus up to ~128 wall placements), so the search tree at depth 3 had millions of nodes.

Solution: We reduced the depth to 2, which limits the search to two half-moves (the AI's own move and the opponent's reply). At depth 2 the AI responds instantly on a standard 9×9 board and still plays a sensible, challenging game. The depth is set as a constructor parameter in MinimaxStrategy, so it can be increased on smaller boards if needed.

Challenge 2: Alpha-beta pruning made a visible difference

Without pruning, the raw Minimax search at depth 2 was noticeably slow when a player still had all 10 walls available, because the wall-move part of the tree is enormous. We measured the difference by temporarily disabling the pruning condition.

Solution: Alpha-beta pruning is active in the final build. The alpha variable tracks the best score the maximising player has found so far; beta tracks the best for the minimiser. Whenever beta ≤ alpha we break out of the loop early. In practice this cut the number of evaluated nodes roughly in half during our testing, which brought depth-2 response time down to well under one frame.

2.4 UNDO ACROSS AI MOVES

Challenge 1: AI playing again immediately after undo

When we first implemented undo in Human vs. AI mode as a single-move revert, pressing Undo would restore the state to after the human's last move. But because it was now the AI's turn again, the AI would calculate and play a new move in the same frame, making undo appear to do nothing from the player's perspective.

Solution: In GameController::undo(), we check whether the game mode is HUMAN_VS_AI. If it is, we pop two moves from undoStack (the human's move and the AI's reply) and call GameState::undoMove() twice. This always returns the game to the state just before the human's last input, so control stays with the human after undoing.

Challenge 2: Redo re-inserting a stale AI move

After undoing two moves, the redo stack contained the human's old move and the AI's old reply. If the human pressed Redo, both would be re-applied. But the human might have wanted to try a different move instead, and the stale AI reply was from a different game position so it could be illegal or nonsensical.

Solution: In GameController::redo() for HUMAN_VS_AI mode, we re-apply only the human's move and then clear the entire redo stack. This means the AI will calculate a fresh response for the new board position, which is the correct behavior. The human cannot redo back into the AI's previous answer.

3. AI ALGORITHM EXPLANATION

3.1 OVERVIEW

Three AI difficulty levels are provided. All three implement the IAIStrategy interface, which has a single method selectMove(state, playerId) returning a Move object. The active strategy is injected into AIPlayer at game start and never changes during a session. All strategies work on a read-only view of the game state; they create internal copies when they need to simulate future moves.

Difficulty	Class	Approach
Easy	RandomStrategy	Picks a move uniformly at random from all legal moves.
Medium	GreedyStrategy	Evaluates all legal moves with a one-ply heuristic and picks the best.
Hard	MinimaxStrategy	Two-ply minimax search with alpha-beta pruning and a BFS-based evaluation.

3.2 EASY — RANDOM STARTEGY

RandomStrategy calls GameState::getValidMovesForPlayer() to get the list of all legal moves, then picks one index at random using a Mersenne Twister (std::mt19937) seeded from std::random_device. There is no evaluation and no lookahead. The AI is intentionally weak and unpredictable, suitable for new players learning the rules or for testing the game logic.

3.3 MEDIUM —GREEDY STRATEGY

GreedyStrategy performs a one-ply (depth-1) search. For every legal move, it applies the move to a copy of the state and scores the result with a heuristic. The move with the highest score is returned.

Heuristic

```
score = (oppBFSDist - myBFSDist) + (myWallsLeft
- oppWallsLeft)
```

myBFSDist is the shortest-path distance (in steps) from the AI's current position to its goal row, computed by PathFinder::shortestPath(). oppBFSDist is the same for the opponent. A distance of 1000 is assigned to any player with no reachable path. The wall differential term rewards the AI for preserving more walls than the opponent.

This heuristic naturally scores pawn advances toward the goal highly (myBFSDist decreases) and scores well-placed walls highly (oppBFSDist increases). Because it is evaluated after the candidate move is applied, the AI considers both types of move on equal footing.

3.4 HARD — MINIMAX WITH ALPHA-BETA PRUNING

MinimaxStrategy uses the minimax algorithm with alpha-beta pruning and a depth limit of 2. The search alternates between maximising the AI's score (the AI's own turn) and minimising it (simulating the opponent's best reply).

Algorithm:

```
int MinimaxStrategy::minimax(GameState& state, int
depth, int alpha, int beta, int playerId) const {
    if (depth == 0 || state.isGameOver()) {
        return evaluate(state, playerId);
    }

    int currentPlayerId =
state.getCurrentPlayer().getId();
    vector<Move> moves =
state.getValidMovesForPlayer(currentPlayerId);
    if (moves.empty()) {
        return evaluate(state, playerId);
    }

    bool maximizing = (currentPlayerId == playerId);
    int bestValue = maximizing ?
numeric_limits<int>::min() :
numeric_limits<int>::max();

    for (const Move& move : moves) {
        GameState copy = state;
        if (!copy.applyMove(move)) {
```

```
            continue;
        }
        int value = minimax(copy, depth - 1, alpha,
beta, playerId);

        if (maximizing) {
            bestValue = max(bestValue, value);
            alpha = max(alpha, value);
        } else {
            bestValue = min(bestValue, value);
            beta = min(beta, value);
        }
        if (beta <= alpha) {
            break;
        }
    }
    return bestValue;
}
```

Alpha-Beta Pruning:

Alpha is the best score the maximiser has already secured; beta is the best score the minimiser has secured. When beta ≤ alpha, the current branch cannot change the final result — the maximiser already has a better option elsewhere and the minimiser already has a better option, so the remaining siblings are skipped. This can reduce the number of evaluated nodes from O(b^d) to roughly O(b^(d/2)) in the best case.

Evaluation Function

```
score = oppBFSDist - myBFSDist
```

The wall differential term used by Greedy is omitted here because the two-ply search already factors in wall impact through the BFS distances of future states. Including it would cause double-counting.

Why Depth 2?

A single Quoridor turn can have up to ~130 legal moves. At depth 3, the search tree can have over two million nodes, making the AI take several seconds to respond. Depth 2 produces a search tree of at most ~130^2 ≈ 17,000 nodes, which the AI evaluates in well under one frame at 60 FPS.

3.5 PATHFINDER

PathFinder provides the BFS utilities used by all AI strategies and by the wall-placement validation in the model:

- **bfs(board, start, goalRow)** — returns true if any path exists from start to goalRow. Used to validate that a wall placement does not strand a player.
- **shortestPath(board, start, goalRow)** — returns the minimum number of steps from start to goalRow (or -1 if none exists). Used as the core metric in Greedy and Minimax evaluations.

4. ASSUMPTION MADE DURING DEVELOPMENT

4.1 BOARD

- Board size can be configured from 5×5 to 12×12 via the menu spinner.
- Player 1 starts at row 0, column size/2 and must reach row size−1. Player 2 starts at row size−1, column size/2 and must reach row 0.
- The wall grid is (size−1)×(size−1); a wall at anchor (r, c) spans cells (r, c) and (r, c+1) for horizontal, or (r, c) and (r+1, c) for vertical.

4.2 SAVE/LOAD

- Save always writes to a fixed file named savegame.qdr in the working directory. There is no multi-slot save system.
- If no save file exists when Load is pressed, the operation fails silently and the HUD shows “Load failed”.
- The undo and redo stacks are saved along with the game state, so the player can undo moves made before saving.

4.3 AI

- In Human vs. AI mode, the human is always Player 1 and the AI is always Player 2.
- The AI computes its move synchronously on the same frame it is triggered. No background threading is used.
- The Minimax search is not time-limited; it always searches to the full depth of 2.

5. DEMO VIDEO

Video link

6. REFERNCES AND RESOURCES USED

6.1 GAME RULES AND BACKGROUND

- Mirko Marchesi. Quoridor — Official Rules. Gigamic, 1997.
- BoardGameGeek. Quoridor entry.
- Quoridor Rules & Strategies Explained (video).

6.2 DESIGN PATTERNS

- Gamma, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994. (Strategy pattern, p. 315.)
- Martin, R. C. Clean Architecture. Prentice Hall, 2017. (MVC separation of concerns.)

6.3 ARTICLES AND GUIDES WITH VISUALIZATION

- Quoridor Strategy Guide with Animated Examples
- Interactive Quoridor Tutorial - Online playable demonstration
- Mathematical Analysis of Quoridor - Blog with interactive visualizations of AI strategy
- Quoridor AI Development Guide - Article series on implementing Quoridor AI with

6.4 OTHER RESOURCES

- Git and GitHub Tutorial.